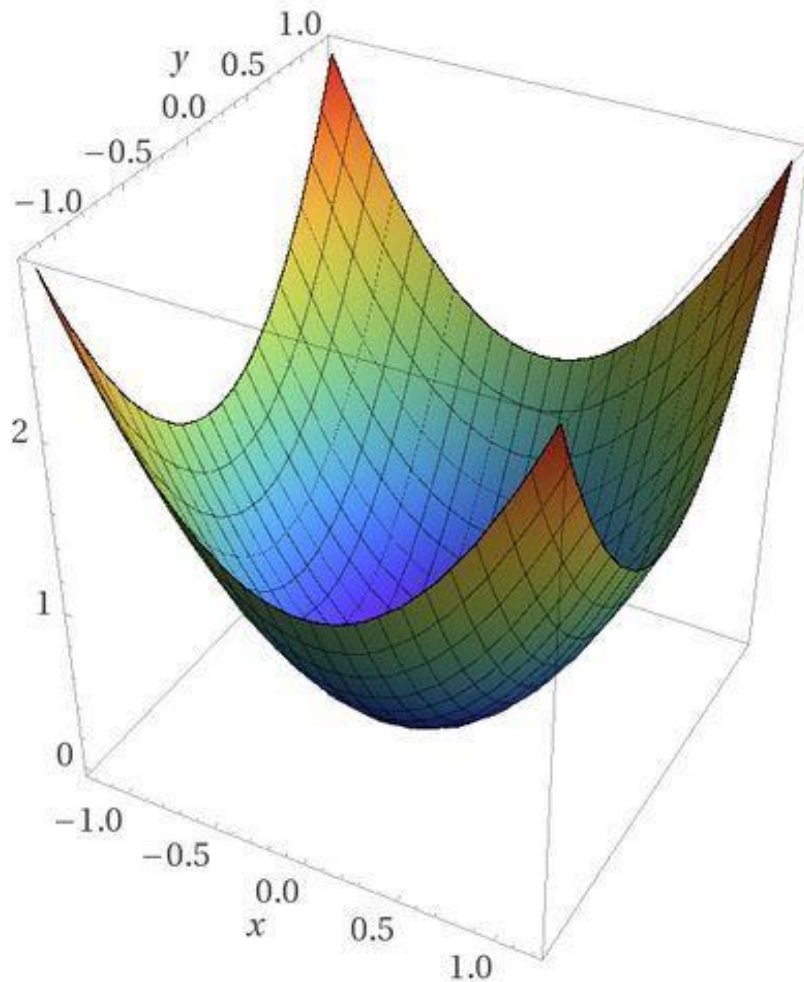


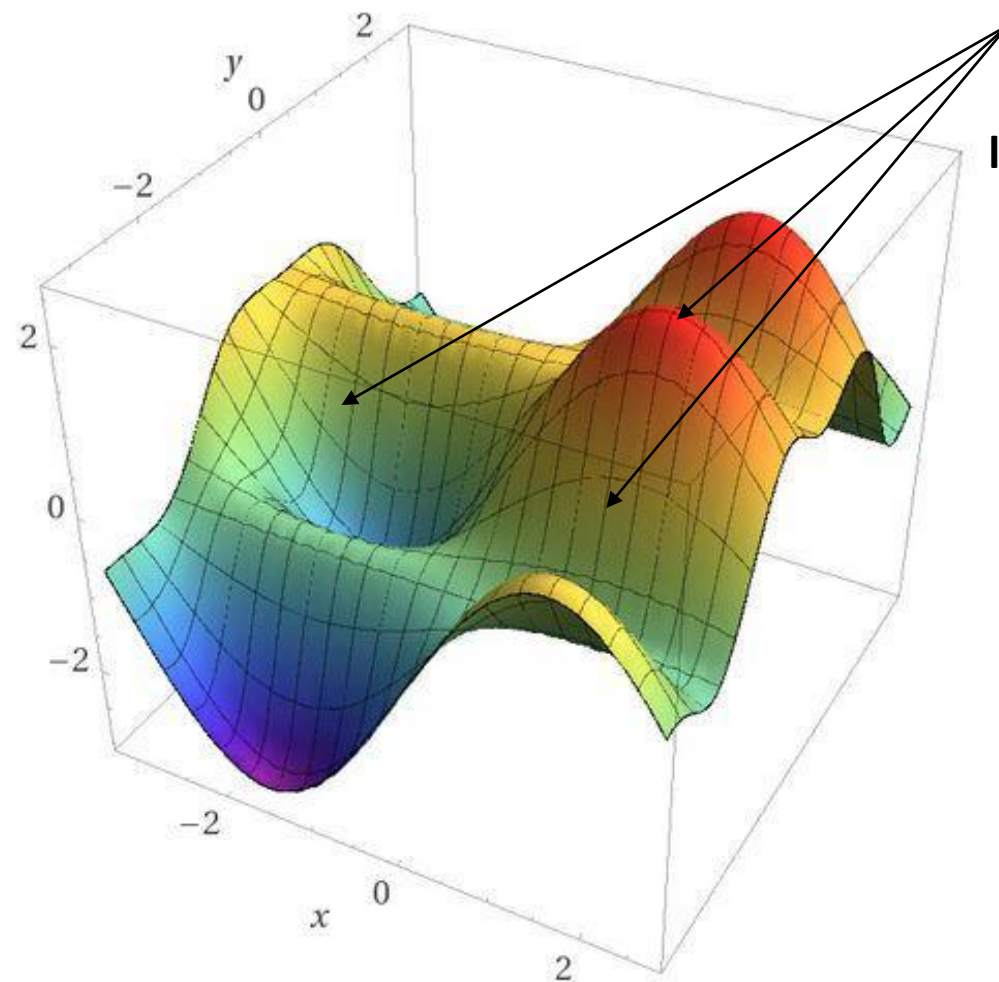
Beyond Backpropagation

Tips and tricks for training deep neural networks

Finding Global Minima: Why is it hard?



Loss space in our expectation

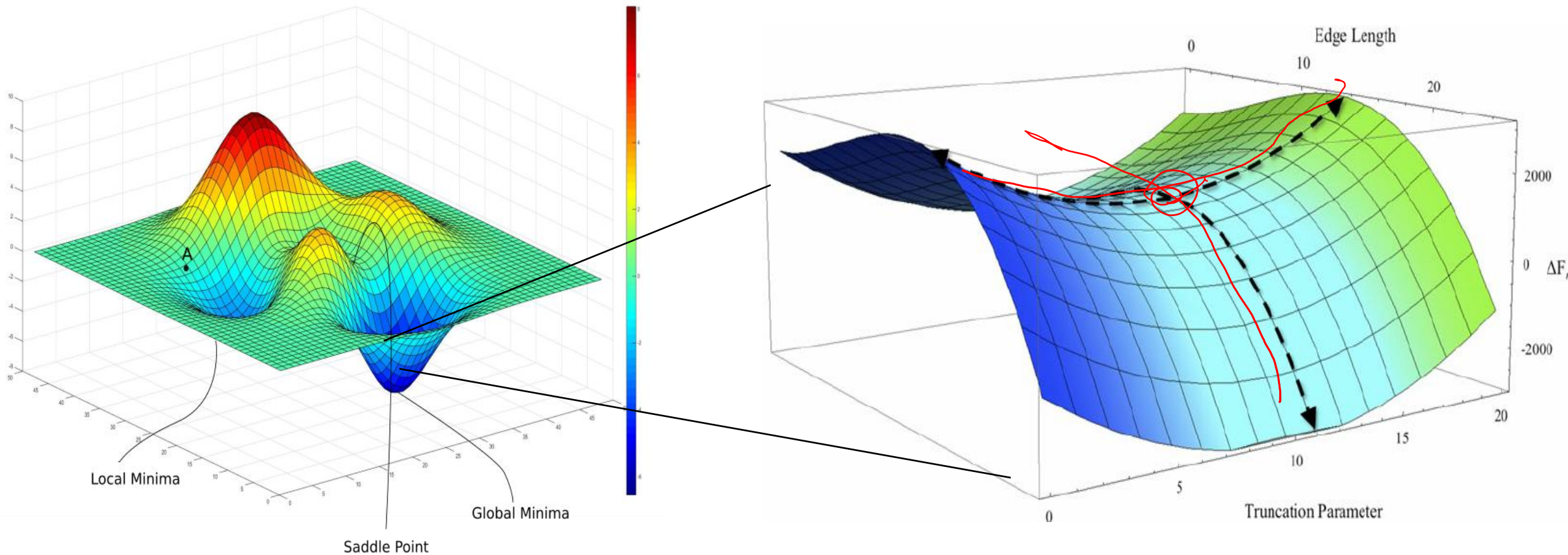


local minima

$$\nabla J = 0$$

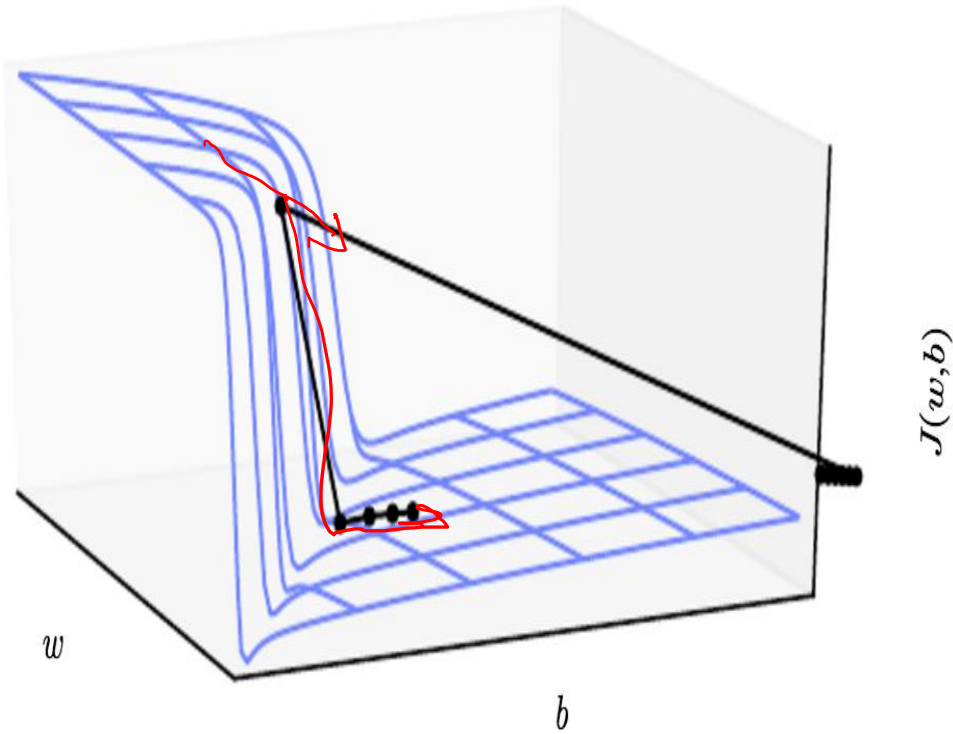
Loss space in reality

Finding Global Minima: Why is it hard?

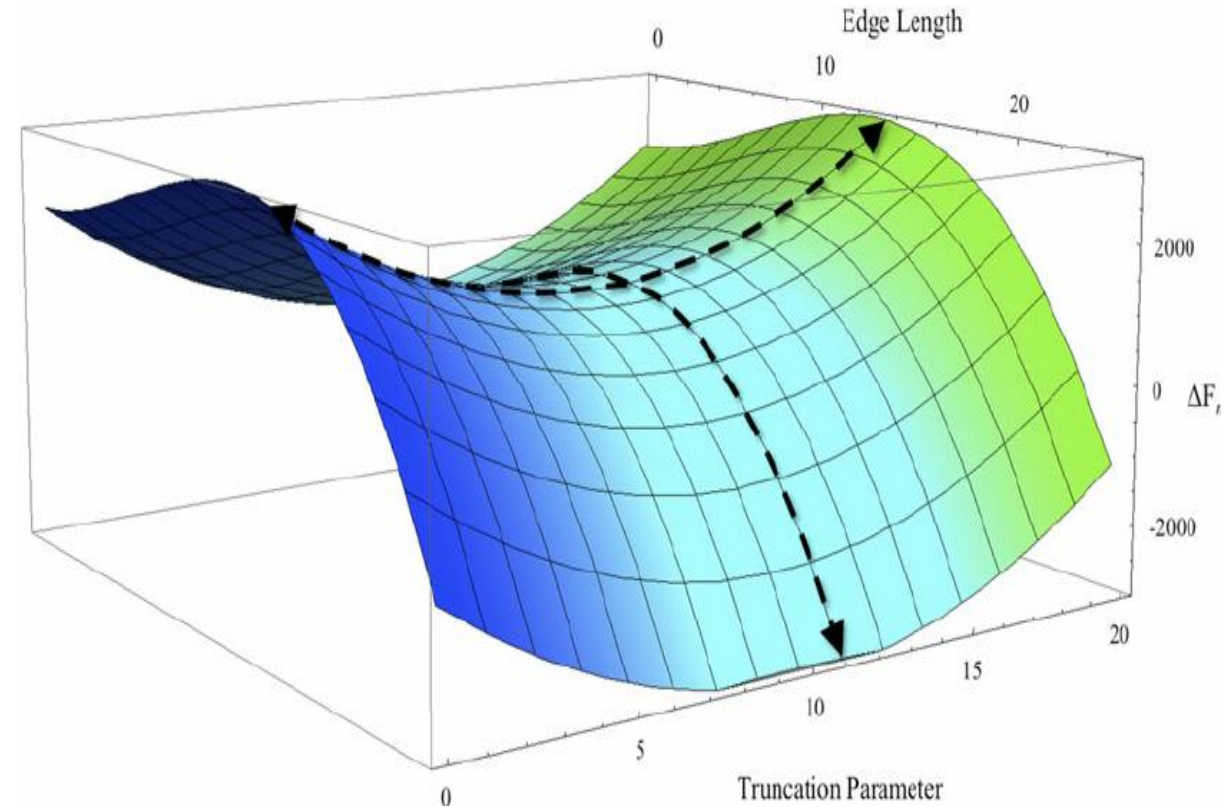


Saddle Point: Local minima in one direction, local maxima in another direction

Finding Global Minima: Why is it hard?



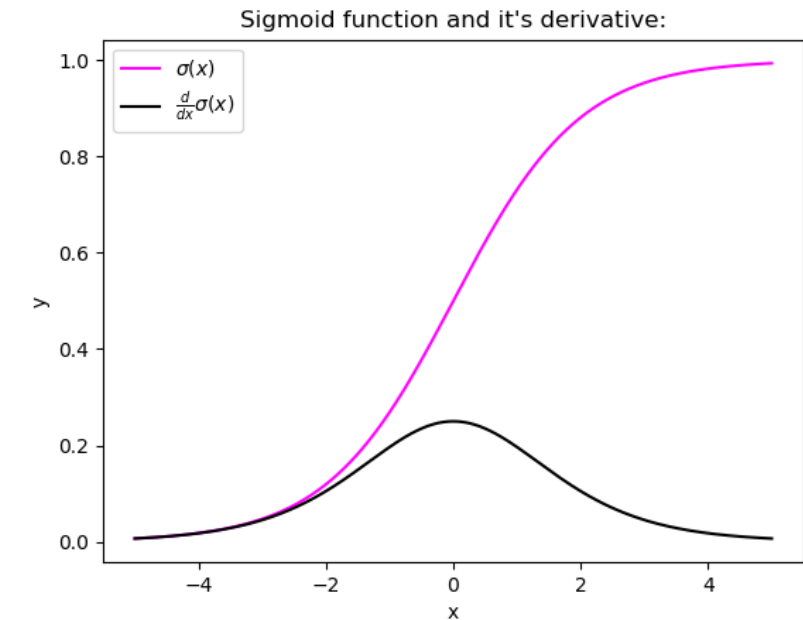
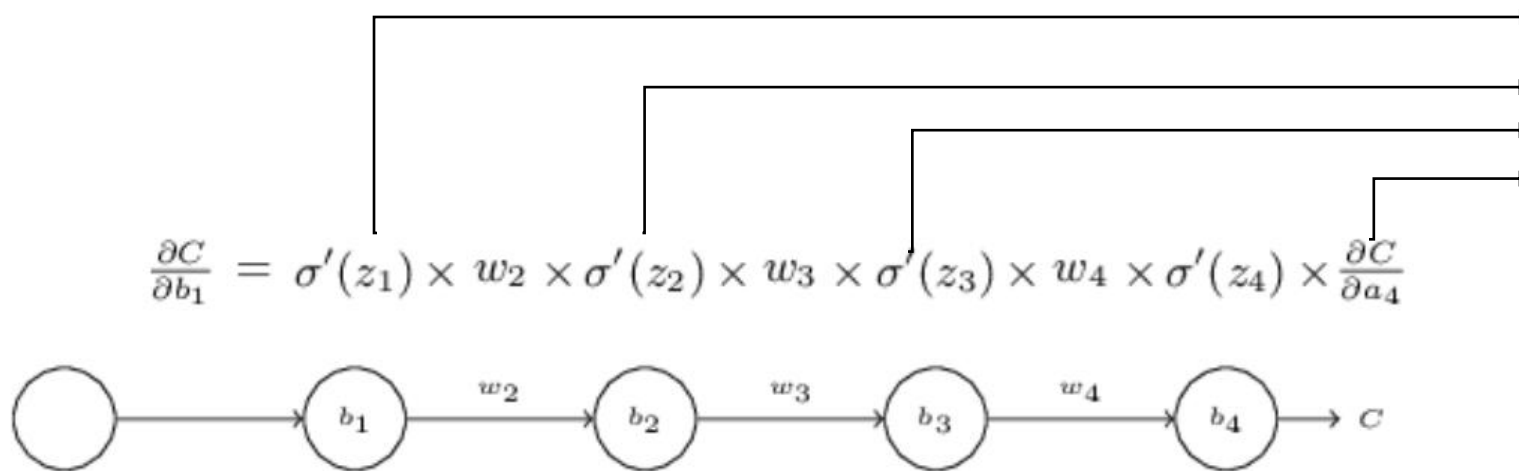
Cliffs : gradient is too high



Plateau : gradient is almost zero

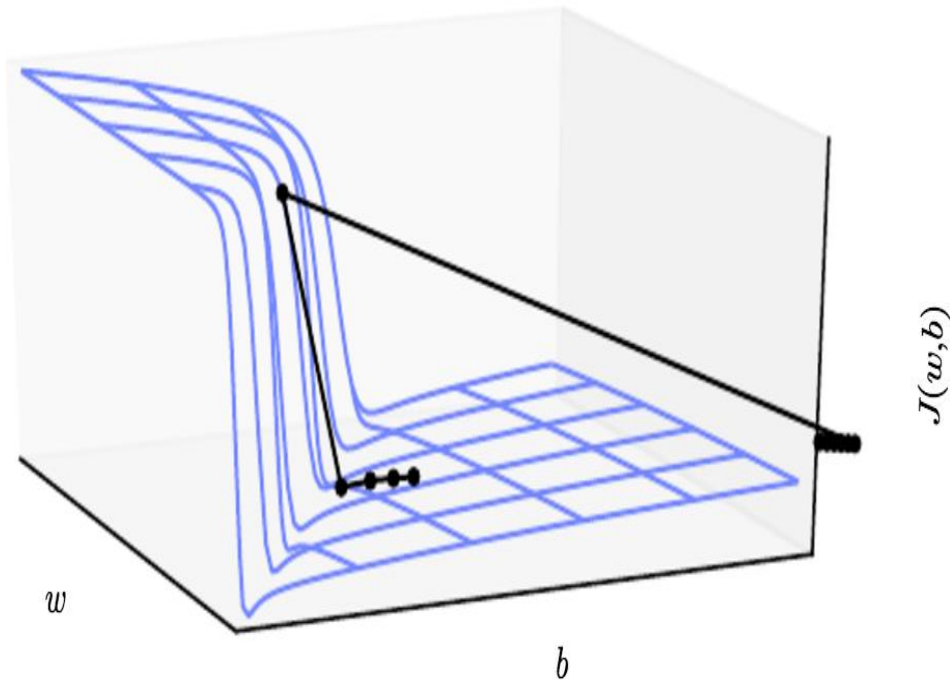
Finding Global Minima: Why is it hard?

The Vanishing Gradient Problem:



$$\left\| \frac{\delta h_i}{\delta h_{i-1}} \right\|_2 < 1$$

Finding Global Minima: Why is it hard?



$$\left\| \frac{\delta h_i}{\delta h_{i-1}} \right\|_2 > 1$$

Solution: Gradient clipping

When gradient is too high, the repetitive multiplication results in exploding gradient

BLANK SLIDE

Solution – I: Better Optimizers

Ideal optimizer:

- Finds minimum fast and reliably well
- Doesn't get stuck in local minima, saddle points, or plateau region

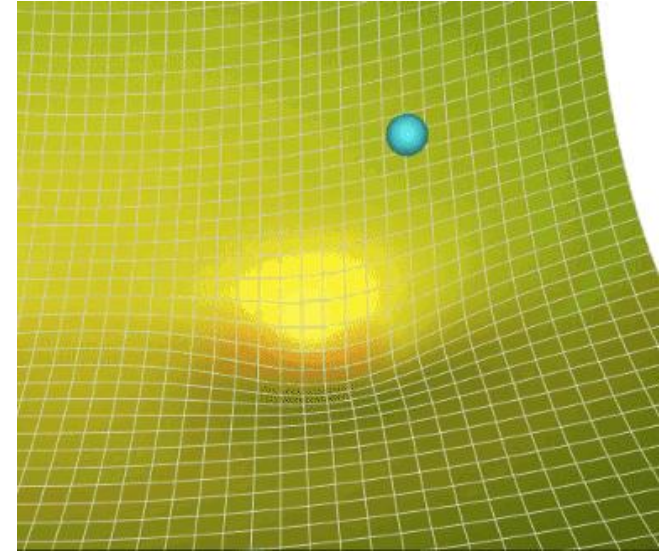
$$w_{t+1} = w_t - \alpha * \frac{\delta L}{\delta w_t}$$

Vanilla Gradient Descent: One step for the entire dataset

Stochastic Gradient Descent: One step for each stochastically chosen sample

Mini-batch Gradient Descent: One step for each mini-batch of samples chosen stochastically

Best of both worlds!



```
1 import torch.optim as optim
2
3 optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0)
```

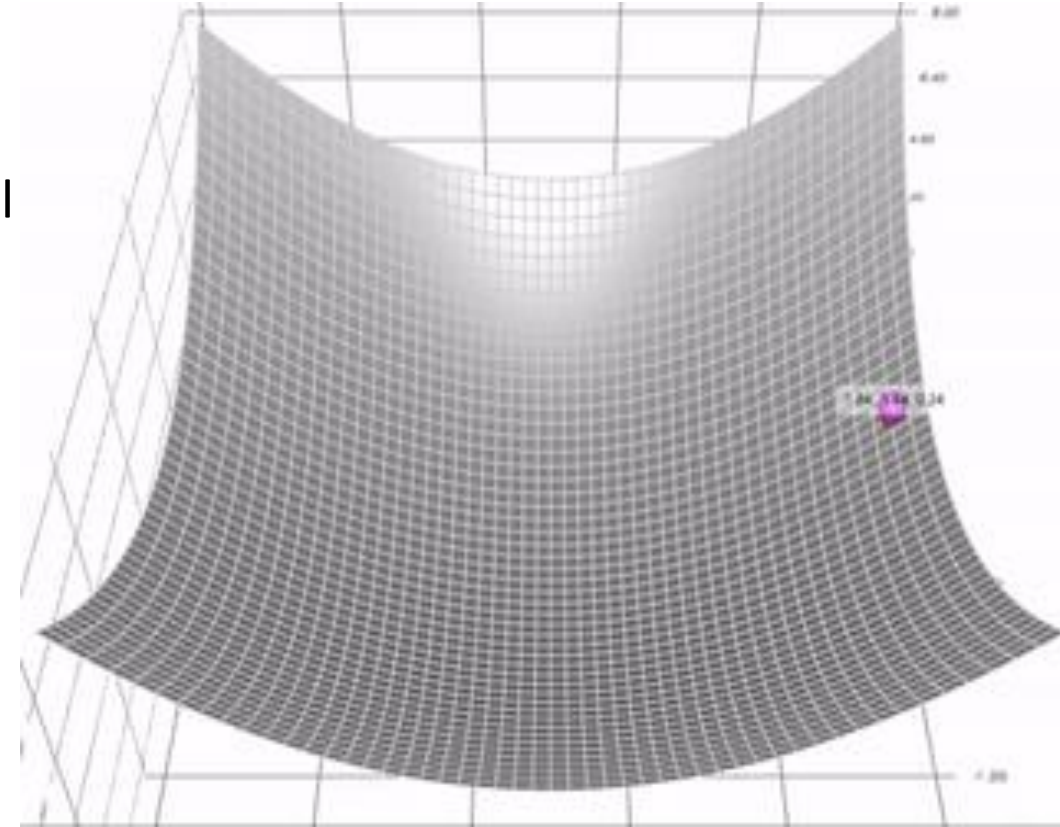
Solution – I: Better Optimizers

Gradient Descent with Momentum

- Imagine a rolling down a ball inside a frictionless bowl
- The ball doesn't stop at the bottom of the surface
- Uses the accumulated momentum to go forward

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta - \gamma v_{t-1})$$
$$\theta = \theta - v_t$$

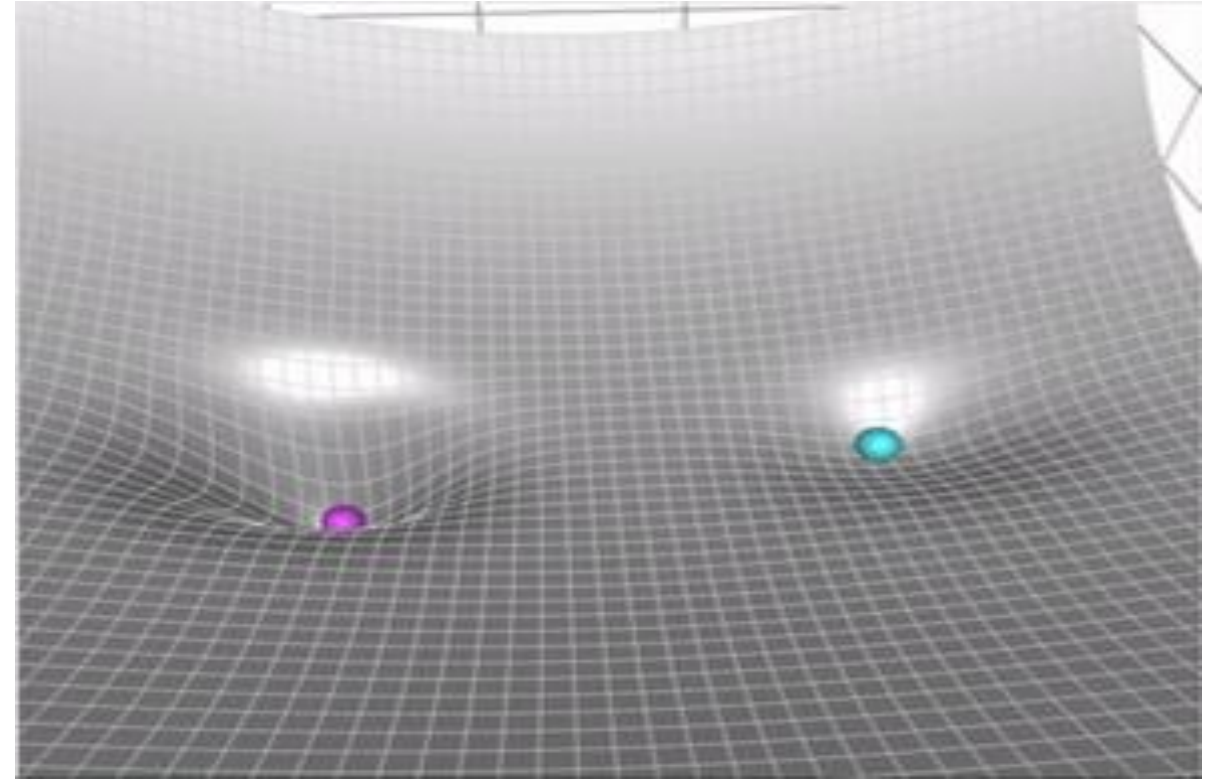
Momentum



Solution – I: Better Optimizers

Gradient Descent with Momentum

Intuitively, this helps us to come out of local minima



$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta - \gamma v_{t-1})$$

$$\theta = \theta - v_t$$

Momentum

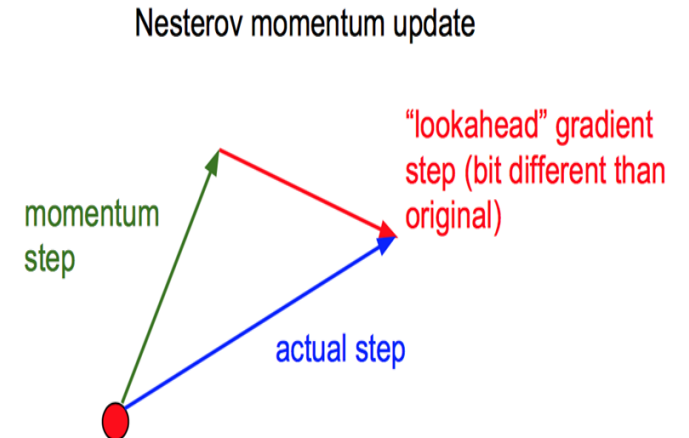
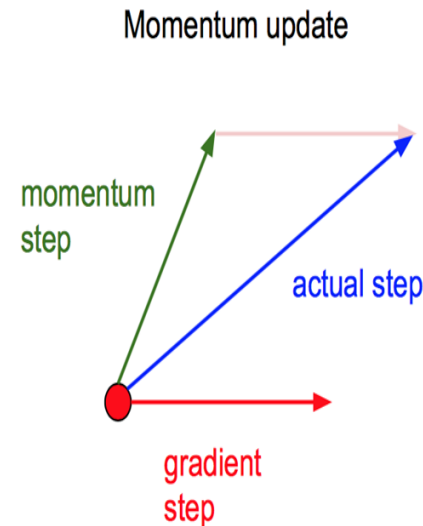
```
1 import torch.optim as optim
2
3 optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

Solution – I: Better Optimizers

Gradient Descent with Nesterov Momentum

- Following the slope blindly is not desirable and optimal
- The ball should predict and slow itself down before going up again

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta - \gamma v_{t-1})$$
$$\theta = \theta - v_t$$



```
1 import torch.optim as optim
2
3 optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=True)
```

Solution – I: Better Optimizers

Other Variants:

- RMSprop
- Adagrad
- **Adam**
- Adadelta
- etc.

```
1 import torch.optim as optim
2
3 optimizer = optim.RMSprop(model.parameters(), lr=0.01, alpha=0.99)
```

```
1 import torch.optim as optim
2
3 optimizer = optim.Adagrad(model.parameters(), lr=0.01)
```

```
1 import torch.optim as optim
2
3 optimizer = optim.Adam(model.parameters(), lr=0.01, betas=(0.9, 0.999))
```

```
1 import torch.optim as optim
2
3 optimizer = optim.Adadelta(model.parameters(), lr=0.01, rho=0.9)
```

Simplified View: Variable Learning Rates for Features/Dimensions

Solution – II: Learning Rate Scheduling

- Adjust the learning rate during training by reducing the learning rate at per predefined scheduled
- Common schedulers –
 - step decay
 - exponential decay
 - cosine decay
 - reduce on plateau
 - etc.

```
1  import torch.optim as optim
2
3
4  scheduler = optim.lr_scheduler.MultiStepLR(optimizer, milestones=[30,80], gamma=0.1)
5  scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.1)
6  scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode = 'min')
7  scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer)
```

Solution – III: Weight Initialization

- All zero initialization
 - Initializing all the weights with zeros leads the neurons to learn the same features during training.
- Random initialization
 - Gaussian
 - Xavier
 - uniform
 - normal
 - Kaiming
 - uniform
 - normal

```
1  import torch
2
3  w = torch.empty(3, 5)
4  torch.nn.init.kaiming_uniform_(w, mode='fan_in', nonlinearity='relu')
```

Summary of Training (Recap)

- Backpropagation Algorithm
 - Gradient Descent
 - Non-Convex Optimization (in NN)
 - Generic Algorithm
- Applicable for
 - MLP
 - CNN
 - RNN
 - A wide class of networks with differentiable units

Summary of Training

- Challenges
 - Non-Convexity, Plateau, Vanishing and Exploding Gradients
- Better Optimizers
 - SGD
 - Momentum
 - Adam
- Learning rate Scheduling
- Better/Smarter Initializations

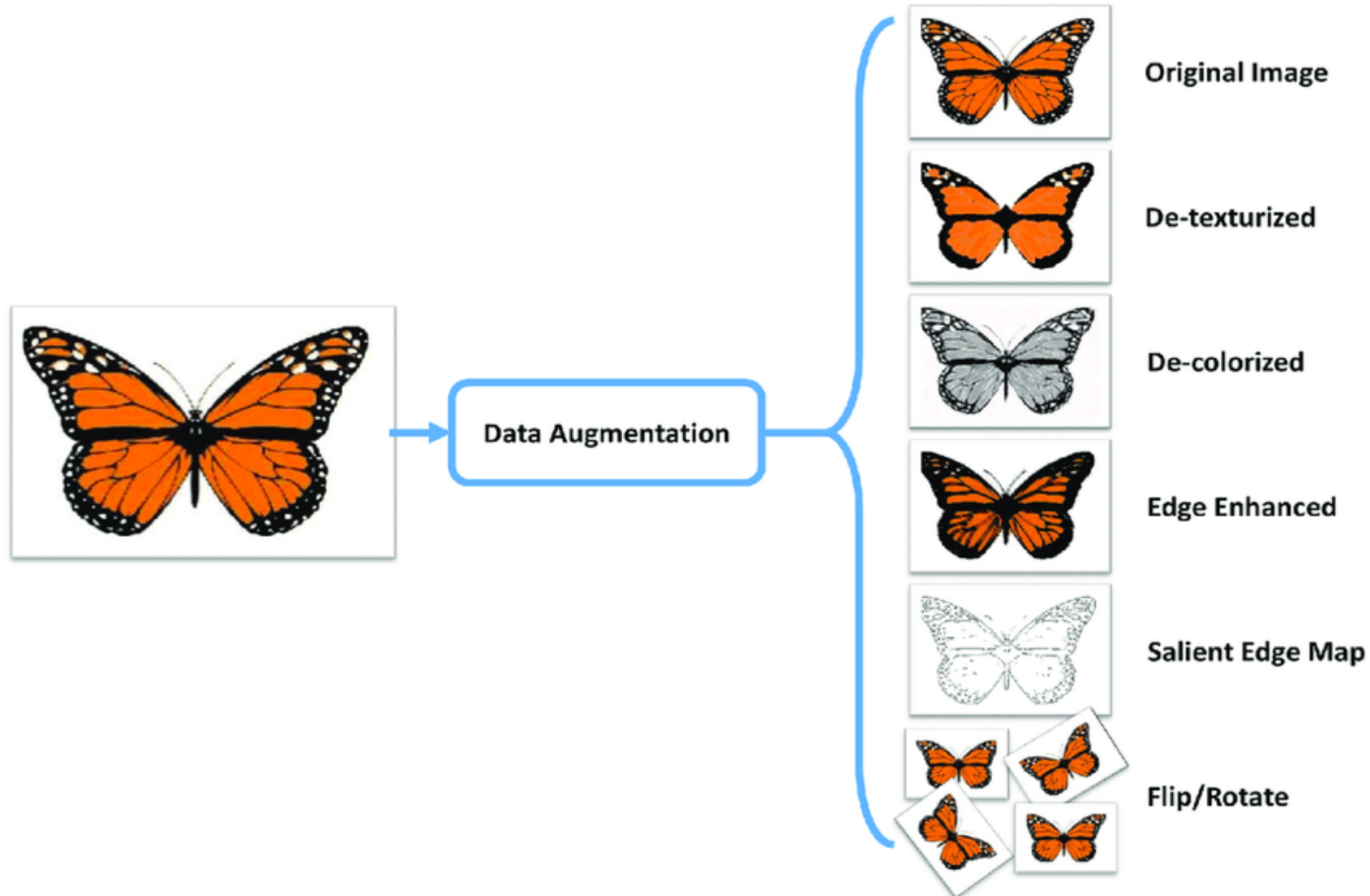
BLANK SLIDE

Regularization of the Network

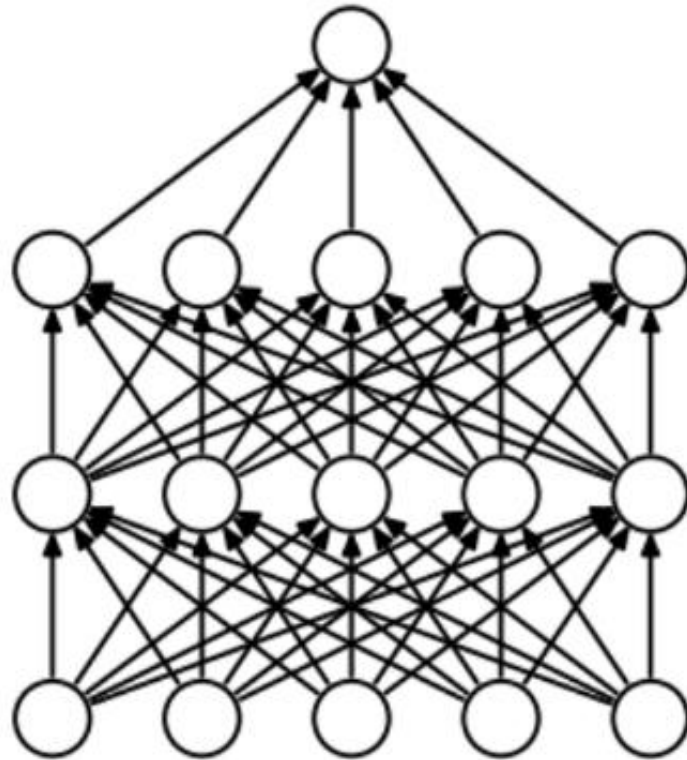
- Input Level
 - Data Augmentation
- Activation level
 - Dropout
 - Dropconnect
- Feature statistics level
 - Batch normalization
 - Layer normalization
 - Group normalization
- Decision level
 - Ensemble
- Constraining network weights
 - ℓ_1 norm, ℓ_2 norm
- Terminating early based of the performance on validation set
 - Early stopping

Regularization at the Input Level

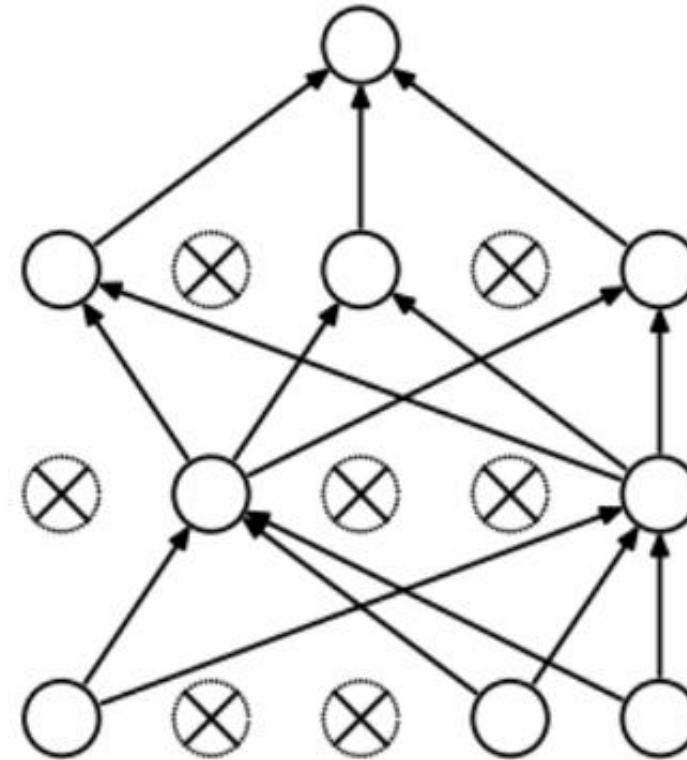
A trick to increase the training data



Dropout



(a) Standard Neural Net

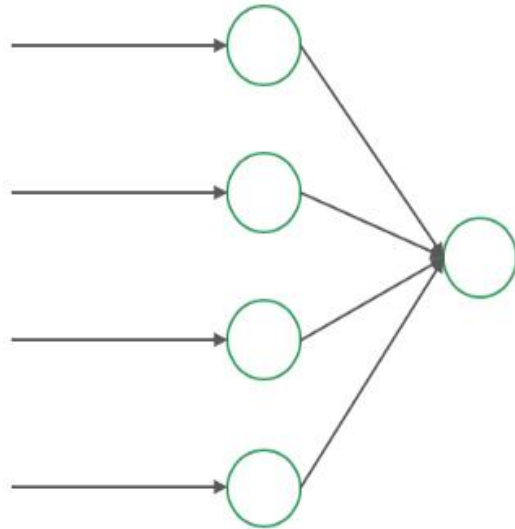


(b) After applying dropout.

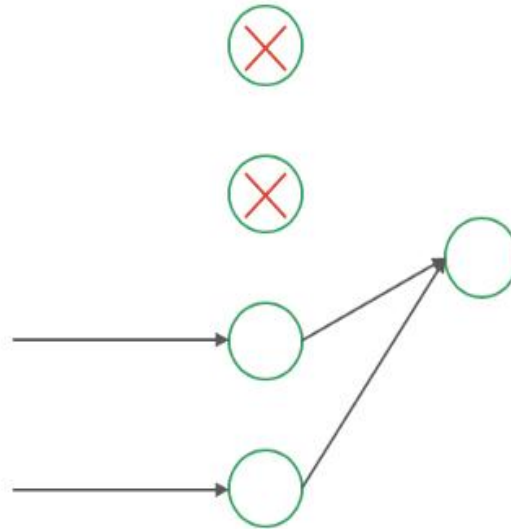
BLANK SLIDE

Regularization at the Activation Level

Dropout:



a. Full network



b. Partial learning of weights over iterations

At each training stage, individual nodes are either dropped out of the net with probability $1-p$ or kept with probability p

Dropout

- Ignore/delete/mask certain neurons while training.
 - Get a simpler network.
 - Eg. Multiply the outputs by 0 or 1 at random.
- Equivalent to creating many neural networks.
- Reduces complex co-adaptations of neurons, since a neuron cannot rely on the presence of particular other neurons.
- Force to learn more robust features that are useful in conjunction with many different random subsets of the other neurons.

Dropout

- Set the output of each hidden neuron to zero with a probability of p (say 0.5).
- The neurons which are “dropped out” in this way do not contribute to the forward pass and do not participate in back propagation.
- Thus neural network samples a different architecture, but all these architectures share weights.
- At test time, scale outputs by probability p .

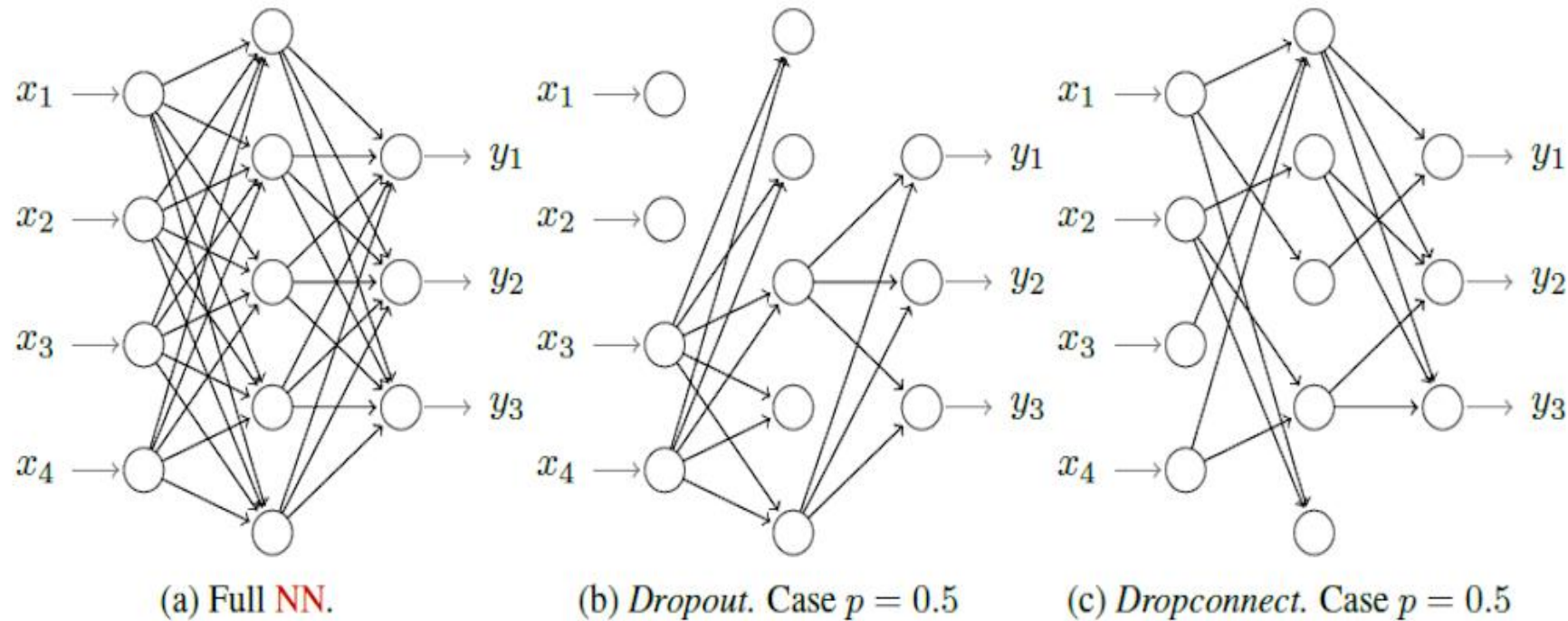
BLANK SLIDE

Drop-Connect

- Drop-connect (Wan et al 2013) is very similar to dropout.
- Randomly deactivates the network weights instead of randomly reducing the neuron activations to zero.
- Performs mask out operations on weights (instead of outputs).

Regularization at the Activation Level

Dropconnect:



At each training stage, disable individual weights (i.e., set them to zero), instead of nodes, so a node can remain partially active

Regularization at the Feature Statistics Level

BatchNorm:

- We normalize the input data before feeding to the NNs
- What about the input distribution at each hidden layer?
 - the distribution changes with iterations
 - internal co-variate shift
 - Solution: Normalize the layer's input over a mini-batch

```
1 import torch
2 import torch.nn as nn
3
4 m = nn.BatchNorm2d(100, affine=False)
5 input = torch.randn(20, 100, 35, 45)
6 output = m(input)
7
```

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Weight Decay, Constraining Weights

- Add a term corresponding to weights into the objective function.
- Smaller the weight (or even zero), the better.

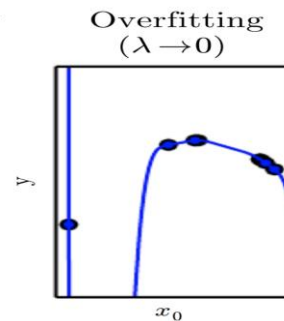
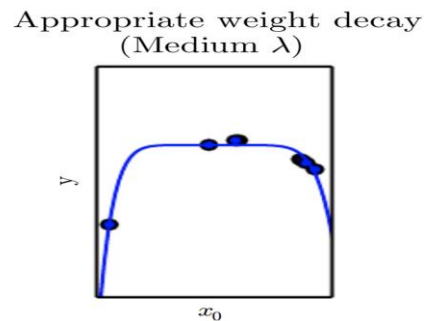
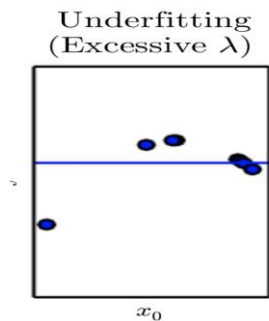
$$J(w) = \mathcal{L} + \lambda ||w||_p$$

L1 Regularization

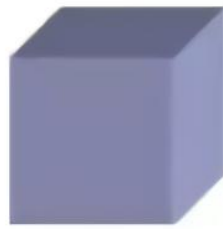
$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^M W_j^2}_{\text{Regularization Term}}$$



Norm and Sparsity



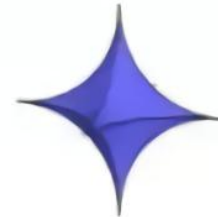
$$p = \infty$$



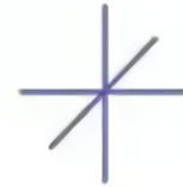
$$p = 2$$



$$p = 1$$

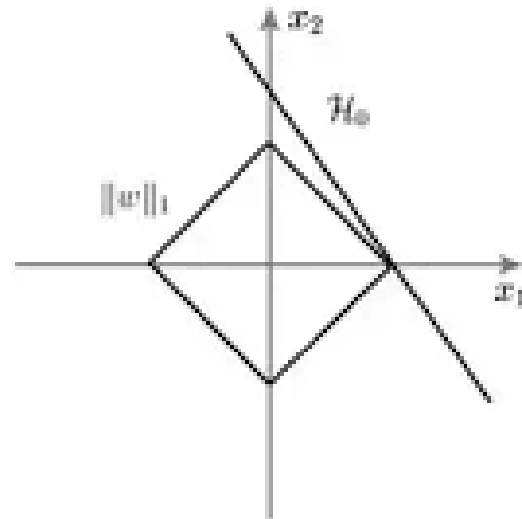


$$0 < p < 1$$

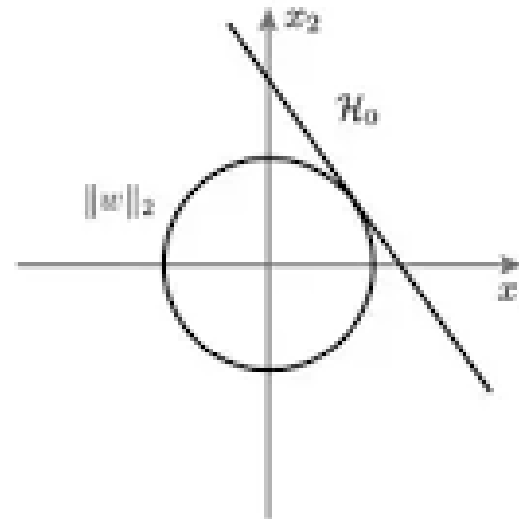


$$p = 0$$

A L1 regularization



B L2 regularization

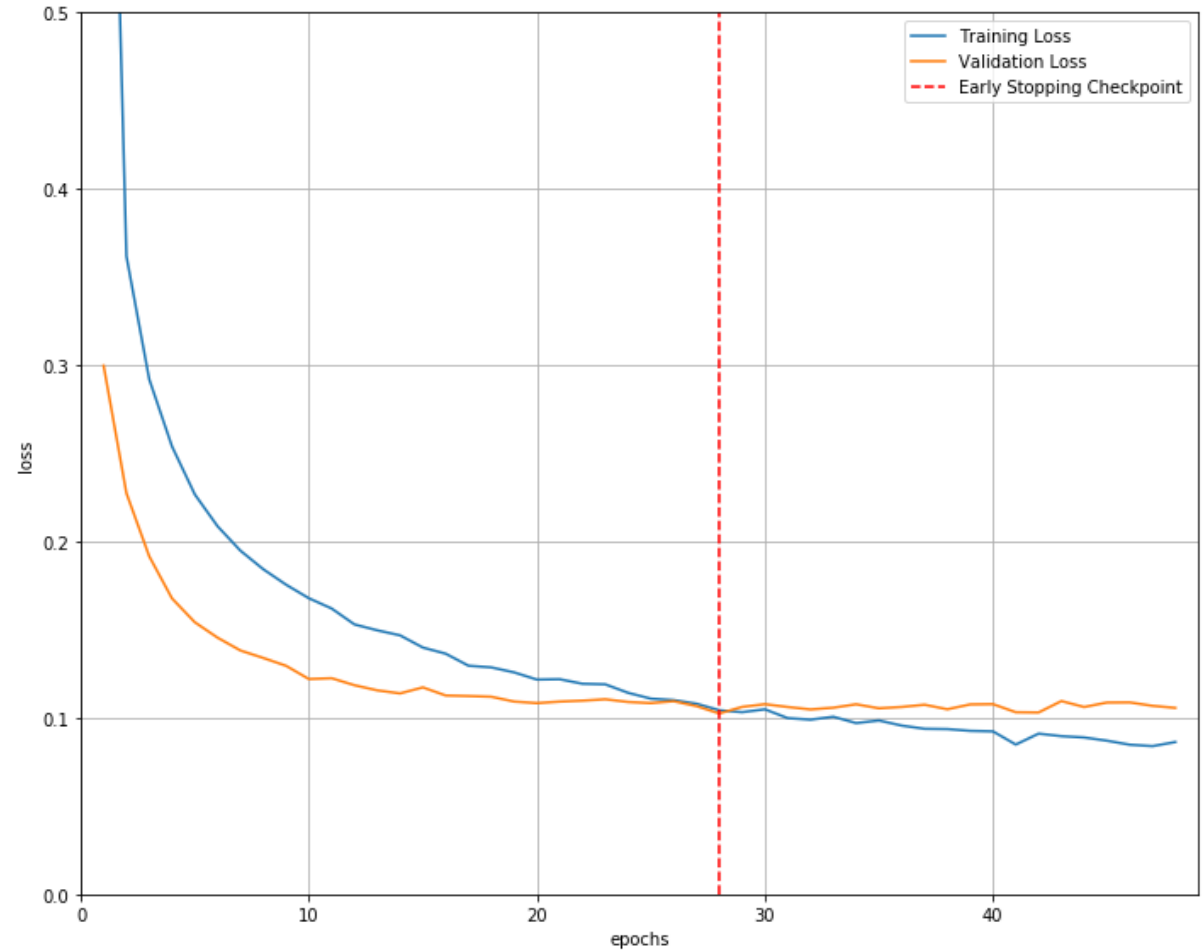


L1 is likely to yield
sparse solutions

BLANK SLIDE

Early Stopping

- Stop the training if validation loss/accuracy not improving
- Avoids overfitting on the training data



Summary

- Data Normalization
- Data Augmentation
- Weight Initialization
- Optimization Algorithms
- Regularizer
- Batch Norm
- Dropout

Many Improvements

- Better Learning/Optimization
- Better Generalization/Regularization
- Better Loss Functions

Thanks!!

Questions?